

Objective of Step 2 (so far)

Goal of Step 2 (partial):

Convert Phase-1 menu items into vectors using Sentence Transformers and store them in a FAISS index, along with proper metadata and mappings.

This enables:

- semantic food search
- similarity-based recommendations
- fast retrieval at scale

⚠️ We are **not** building APIs yet.

⚠️ Only **offline index building** so far.

New Service Introduced: index-service

Why a separate service?

- Indexing is **offline** / batch
 - Retrieval is **online** / API
 - Clean separation = production-grade design
-

Folder Structure Created (Step 2)

```
services/  
  index-service/  
    requirements.txt  
    README.md    (optional)  
    src/  
      __init__.py  
      config.py  
      db.py  
      index_utils.py  
      build_index.py
```

Each file has a **very specific role**.

requirements.txt

File

services/index-service/requirements.txt

Contents (important libs)

- sentence-transformers
- faiss-cpu
- numpy
- psycpg2-binary
- python-dotenv

Why this file exists

- Isolates index dependencies from other services
- Makes index-service reproducible
- Ensures FAISS + embeddings work together

Concept learned

Indexing stack $\neq$ API stack $\neq$ LLM stack

Each service should have its own dependencies.

2 config.py

File

services/index-service/src/config.py

What it stores

- Postgres connection details
- Which table to read from:

```
MENU_ITEMS_TABLE = "phase1_menu_items"
```

- Embedding model name:

```
EMBEDDING_MODEL_NAME = "all-MiniLM-L6-v2"
```

- Index output directory

Why this file exists

- Central place for configuration
- No hardcoded values inside logic
- Easy to switch:

- Phase 1 → Phase 2
- embedding model → better model later

Concept learned

All environment decisions belong in config, not in logic

3 db.py

File

services/index-service/src/db.py

What it does

- Creates a reusable Postgres connection function:

```
get_conn()
```

Why this file exists

- Avoids duplicating DB connection code
- Keeps DB logic separate from indexing logic

Concept learned

One responsibility per file (SRP principle)

4 index_utils.py

File

services/index-service/src/index_utils.py

What it provides

Utility/helper functions:

- `generate_index_version()`
→ creates version like 20240208_2215
- `ensure_dir()`
→ ensures index folder exists

- `save_json()`
→ safely saves mapping & metadata

Why this file exists

- Keeps `build_index.py` clean
- Centralizes reusable helpers
- Makes index versioning explicit

Concept learned

Versioning is not optional in ML systems

5 Index Versioning (Conceptual but implemented)

What is index versioning?

Every FAISS build is tagged with:

`index_version` = YYYYMMDD_HHMM

Example:

```
faiss_20240208_2215.index  
mapping_20240208_2215.json  
metadata_20240208_2215.json
```

Why versioning is critical

- Track which data + model produced an index
- Allow rollback
- Avoid "mystery index" bugs
- Industry-standard ML practice

Concept learned

An index is a **deployable artifact**, not a temp file.

6 `build_index.py` (Core of Step 2)

File

```
services/index-service/src/build_index.py
```

This is the **heart of Step 2**.

What this script does (high level)

1. Reads **Phase-1 menu items** from Postgres
 2. Loads **Sentence Transformer model**
 3. Converts `embedding_text` → vectors
 4. Builds a **FAISS index**
 5. Saves:
 - FAISS index file
 - `vector_id` → `item_id` mapping
 - index metadata (JSON)
-

Important parts inside `build_index.py`

◆ Fetch data

```
SELECT item_id, embedding_text FROM phase1_menu_items
```

Why:

- `embedding_text` = semantic meaning
 - `item_id` = link back to DB
-

◆ Sentence Transformer usage

```
model = SentenceTransformer("all-MiniLM-L6-v2") embeddings = model.encode(texts,  
normalize_embeddings=True)
```

Why:

- Converts text → dense vectors
 - Normalization enables cosine similarity
-

◆ FAISS index creation

```
index = faiss.IndexFlatIP(dim) index.add(embeddings)
```

Why:

- IndexFlatIP = inner product
 - With normalized vectors → cosine similarity
 - Simple, fast, perfect for Phase 1
-

◆ Mapping creation

`vector_id → item_id`

Why:

- FAISS only returns vector IDs
 - Mapping reconnects FAISS → Postgres row
-

◆ Metadata saving

```
{ "index_version": "...", "embedding_model": "...", "item_count": 268686, "dimension": 384, "phase": "phase1", "similarity": "cosine" }
```

Why:

- Observability
 - Debugging
 - Interview-grade system maturity
-

7 `__init__.py` (Small but important)

File

`services/index-service/src/__init__.py`

Why it matters

- Makes `src` a Python package
- Required for:

```
python -m src.build_index
```

- Missing this caused the silent failure you saw

Concept learned

Python packaging issues can silently break ML pipelines.

Key Concepts Learned in Step 2 (write these)

- Sentence Transformers → text → meaning vectors
 - FAISS → fast similarity search over vectors
 - Indexing is **offline**, retrieval is **online**
 - Normalize embeddings when using cosine similarity
 - Always version ML artifacts
 - Store vector_id → item_id mapping
 - Keep config separate from logic
-

One-line mental summary (Step 2 so far)

Phase-1 menu items → embedding_text → Sentence Transformer → vectors → FAISS index → versioned + metadata-tracked artifact.

What is not done yet in Step 2

- ✗ Retrieval/search test
- ✗ FAISS loading & querying
- ✗ API layer

Those come next.